Systems Programming Project 2026

Do not post solution of this assignment online! This is considered plagiarism by the exam board (this holds for all courses at the VU unless specified otherwise). If you store your repository at github, make it private!!

Goal of this course

The aim of this course is to give you experience with programming a larger project. This larger project involves some of the concepts you became familiar with during the first year of the Bachelor Computer Science, allowing you to put into practice what you have learned.

The course in a nutshell

In this course you will implement an interpreter which executes IJVM bytecode, in the programming language C. The assignment is split into several smaller parts that build up to the final deliverable.

Before starting on the project

First follow the Installation instructions.

And read the following: * VScode guide * Introduction to C * C - Style guidelines * Introduction to the IJVM * Skeleton usage

The project

The project is split into five main chapters. Each chapter builds on the previous chapters and introduces new tasks. We have ranked the difficulty of the chapters to help you better manage your time. Of course, these are just indications, your experience might be different.

Chapter	Difficulty
Binaries, dreaded Binaries!	
Stack up!	
Controlling the Flow: the GOTO solution!	
Local variables: Artisan and Organic!	
Call yourself a method!	
(Optional) Even more stuff	

Grading:

- Your program **must** pass all the **basic** tests. There are a total of 5 basic tests and each basic test is worth 0.6 points. (3 points)
- Your program **must** pass at least 5 of the **advanced** tests. There are a total of 8 advanced tests and each advanced test is worth 0.5 points. (4 points)
- You must pass all JAS assignments (each before its deadline) on canvas to pass the course.
- You must have passed in the IJVM Basics assignment to pass the course.
- If you pass all the basic tests and at least 5 of the advanced tests, your program will be graded on style and general impression. (0.5 point)
- You can achieve a higher grade by implementing additional functionality (3.5 points). **Note:** you are only eligible for these points if you pass at least 5 out of the 8 advanced tests.
- Naturally, your final grade is **capped at a 10**.

You need at least **5.5 points** to pass the course. **Note:** It is ***NOT*** possible to pass by style points.

All submitted code, including comments, must be in English.

You can compute your grade (excluding bonus and style points) by doing `make grade` (after following the installation instructions).

You must pass the digital exam to pass the course. See canvas for details.

Handing in

To hand in your assignment use `make zip` which creates a file called `dist.zip` in your project directory. Hand this in using codegrade. You can drag this zip from the vscode list of files to codegrade if you position the windows side by side.

Of course, you can also find this file using your file explorer. On windows, the file is (probably) at Linux -> /home//sypp-skeleton/dist.zip (path : `\\wsl$/home/<username>/sypp-skeleton/dist.zip`

- If you ever get the situation where a test fails on codegrade but succeeds on your system, run `make testsanitizers`. This will use the Clang sanitizers, which instrument your code to detect behaviour which is not defined in the C standard such as array indexing out of bounds, use of uninitialized variables or reliance on an undefined order. We recommend you always use the sanitizers before submitting to codegrade.

Your program has to compile and work on Ubuntu 20.04 LTS x86-64 with Clang. Check the output of our submission system to make sure that your program compiles on the target environment.

Plagiarism and a note on doing the assignment yourself

Do not copy code from each other or from the internet!

This is an individual programming assignment, which means that you have to program the assignment by yourself. It is not sufficient to understand the program you hand in, you **must** have programmed it yourself.

It is allowed to discuss the assignments and solutions broadly.

You are allowed to get help from a TA or a fellow student that already solved it if you are stuck. It is then:

- OK to show your (not-working) code to them and discuss what is wrong.
- NOT OK to look at their (working) code. The reason here is that the former leads to you understanding what is wrong with your own code, while the latter likely leads to you submitting the same solution as your fellow student.

As soon as you are unstuck, please continue working individually. It is not allowed to program together with a friend/parent/tutor, as it is then impossible to see what can be attributed to you.

We automatically check your code for similarities with the submissions of other students and known solutions.

If you are interacting with an LLM, the same rules hold: you must program the assignment yourself. It is OK to discuss the assignment with an LLM and to ask for debugging help, but ***not*** to let it generate code for you.

Do not post your solutions online! This is not allowed as sharing solutions is also considered fraud by the exam committee (fraud is defined as any action that makes fair assessment impossible).

Please refer to the Teaching and Examination Regulations and The Examination Board's Rules and Guidelines for additional information.

0 - Introduction to the IJVM

> "However, as every parent of a small child knows, converting a large object into small fragments is considerably easier than the reverse process."
>
> Andrew S. Tanenbaum

Like x86, ARM and RISC-V which were discussed in the course Computer Organization, the Java Virtual Machine (JVM) is an Instruction Set Architecture (ISA). In contrast to x86, ARM and RISC-V, the JVM ISA is not commonly implemented in

hardware (but such implementations do exist), but in software. The benefit of this setup is portability: in contrast to say x86 binaries, JVM binaries can be run on any machine with an JVM implementation.

In this course you will implement a variant of the JVM called the Integer Java Virtual Machine (IJVM), as presented in the book ***Structured Computer Organization*** by Andrew S. Tanenbaum. The IJVM instruction set is a subset of the Java Virtual Machine instruction set, containing only operations on integers. This eliminates much of the complexity added by different types.

The IJVM is a stack-based architecture, meaning that (almost) all operations are performed on a stack. This is in contrast to architectures such as Intel x86 which mainly performs its operations on registers. One main benefit of a stack-based architecture is that the instruction set is simpler, since the operations do not need a source and destination operands.

Please make sure you read the chapter on the IJVM from the book ***Structured Computer Organization*** and C - Introduction before continuing.

Tasks

By the end of the course you will have a fully functional IJVM emulator that can execute IJVM bytecode. To achieve this we split the project into five tasks:

1. Parsing the binary.
2. Implementing basic stack manipulation.
3. Implementing control flow.
4. Implementing operations on local variables.
5. Implementing method calls.

By implementing each task and thoroughly testing it, you will avoid common pitfalls that become hard to debug at a later stage.

Note that our tests are pretty comprehensive, but they do not guarantee that you did not do anything wrong. Hence, it may be possible that a test in, say, task 5 fails because you made an error in the functionality of task 3.

The IJVM instruction set

The IJVM architecture has the following instructions: (in general, all arguments are signed, unless they are indices of local variables or constants.)

OpCode	Instruction	Args	Description
0x10	BIPUSH	byte	Push a (signed) byte onto the stack
0x59	DUP	N/A	Push a copy of the top word onto the stack
0xFE	ERR	N/A	Print an error message to the machine output and halt the

OpCode	Instruction	Args	Description
			emulator
0xA7	GOTO	short	Increment the program counter by a (signed) short
0xFF	HALT	N/A	Halt the emulator
0x60	IADD	N/A	Pop two words from the stack and push their sum
0x7E	IAND	N/A	Pop two words from the stack and push their bitwise AND
0x99	IFEQ	short	Pop a word from the stack and branch if it equals zero
0x9B	IFLT	short	Pop a word from the stack and branch if it is less than zero
0x9F	IF_ICMPEQ	short	Pop two words from the stack and branch if they are equal
0x84	IINC	byte byte	Increment a local variable by a value. The first byte is the (unsigned) index of the local variable. The second byte is the (signed) value.
0x15	ILOAD	byte	Push the local variable with (unsigned) index byte onto the stack
0xFC	IN	N/A	Read a character from the machine input and push it onto the stack. If no character is available, push 0 instead.
0xB6	INVOKEVIRTUAL	short	Invoke a method
0xB0	IOR	N/A	Pop two words from the stack and push their bitwise OR

OpCode	Instruction	Args	Description
0xAC	IRETURN	N/A	Return from the current method
0x36	ISTORE	byte	Pop a word from the stack and store it in the local variable with (unsigned) index byte
0x64	ISUB	N/A	Pop two words from the stack and push their difference **Note:** subtract the top word from the second to top word
0x13	LDC_W	short	Push the constant with index (unsigned) short from the constant pool onto the stack
0x00	NOP	N/A	Do nothing
0xFD	OUT	N/A	Pop a word from the stack and print it to the machine output **Note:** print a character
0x57	POP	N/A	Pop a word from the stack
0x5F	SWAP	N/A	Swap the two top words on the stack
0xC4	WIDE	N/A	Prefix instruction; the next instruction has a 16-bit (unsigned) index. This can be prefixed to ILOAD, ISTORE and IINC. This only changes the size of the index to 16 bits, the constant of IINC remains 8 bits.

Example

An IJVM program is specified in JAS (iJvm Assembly Language), for example:

```
.constant
  a -1
  b 2
  c 3
.end-constant

.main
              // state of the stack
  BIPUSH 112 // 112,
  DUP        // 112, 112
  BIPUSH -1  // 112, 112, -1
  IADD       // 112, 111
  DUP        // 112, 111, 111
  DUP        // 112, 111, 111, 111
  BIPUSH  1  // 112, 111, 111, 111, 1
  ISUB       // 112, 111, 111, 110
  OUT        // 112, 111, 111
  OUT        // 112, 111
  OUT        // 112
  OUT        //
.end-main
```

This program will print “noop”: the ASCII characters for (110, 111, 111, 112). This program also features 3 constants a,b and c, which are currently not used but could be loaded on to the stack using **LDC_W**.

The assembler (we use goJASM for this course) translates such JAS files into IJVM binaries. In this binary, the textual instructions are replaced by their opcodes, textual representations of numbers are replaced by binary numbers and names are replaced by numbers (for example, the constant “a” becomes the first constant 0, “b” becomes 1 etc.). (This is not required, but if you ever want to compile your own JAS files to IJVM binaries, you can install goJASM with `make tools`, which will install the `goJASM` binary in the `tools` directory.) Your task now is to load in such an IJVM file and run the program by implementing the instructions.

Web-based implementations

To try out IJVM files, understand how they work and to verify that your implementation works correctly, there are two web-based reference implementations of the IJVM, both made by former students. * WebJVM by Jur van der Berg, which provides a fast implementation, shows the disassembled jas file and where you are in it and has breakpoints. * IJVMore by Elias Groot provides and implementation that shows the binary instead of the disassembled JAS file, provided snapshots and scripting abilites.

Try them out using the example.ijvm of the JAS above.

More examples

As an example with a loop consider the following (ijvm file):

```
.main
   // prints '1' 9 times
L1:
   BIPUSH 0xA // stack: [10]
L2:
   BIPUSH 0x1 // stack: [i,1]
   ISUB       // stack: [i-1]
   DUP        // stack: [i-1,i-1]
   IFEQ END   // stack: [i-1], Jump to end if zero
   BIPUSH 0x31 // stack: [i-1,0x31]
   OUT         // prints '1' (ascii: 0x31)
   GOTO L2     // // Repeat loop
END:
   HALT
.end-main
```

As a more elaborate example, consider the following C program:

```c
int n = 3;
int c = 0;
while (n != 1) {
    c = c + 1;
    if (n % 2 == 0) {
        int divcounter = 0; // compute n / 2 without division instruction
        while (n != 0) {
            n = n - 2;
            divcounter= divcounter + 1;
        }
    } else {
        n = n + n + n + 1; // n * 3 + 1 without multiplication
    }
}
printf("%c",c); // prints 'h' the letter with index 7 (length of collatz sequence)
```

This program computes the length of the Collatz sequence for 9 (= 19). (In case you are wondering what this means, it does not matter for this example, but you can read Wikipedia or watch the first couple minutes of this video.) This program does not use the division operator `/` as it not available in the IJVM instruction set. Instead it computes `n / 2` by counting how often 2 can be subtracted from `n`.

The following JAS code is equivalent to the above C program. This JAS code is used for task 3, so there are no methods or local variables yet. You can try it out in the web-implementations using the ijvm file.

```
.main
BIPUSH 0x00 // counter, c
```

```
BIPUSH 3  /// c, n
LOOP: // c, n
    // check if == 1
        DUP         // c, n, n
        BIPUSH 0x01 // c, n, n, 1
        IF_ICMPEQ DONE // c, n
    // add 1 to counter
        SWAP        // n, c
        BIPUSH 0x01 // n, c, 1
        IADD        // n, c=c+1
        SWAP        // c, n
    // check if even or odd
        DUP         // c, n, n
        BIPUSH 0x01 // c, n, n, 1
        IAND        // c, n, n & 1
        IFEQ EVEN   // c, n
ODD:
    // 3n+1
        DUP
        DUP
        IADD
        IADD
        BIPUSH 0x01
        IADD
        GOTO LOOP
EVEN: // n / 2
      // No division or shift instruction. Instead count how
      // of we can subtract subtract 2
      BIPUSH 1 // divcounter, d . Stack : c,n, d
DIV2: SWAP         // c, d, n
      BIPUSH 0x02  // c, d, n, 2
      ISUB         // c, d, n=n - 2
      DUP          // c, d, n, n
      IFEQ DIVDONE // c, d, n
      SWAP         // c, n, d
      BIPUSH 0x01  // c, n, d, 1
      IADD         // c, n, d=d+1
      GOTO DIV2
DIVDONE:           // c , d ,n
      POP          // c, d
      GOTO LOOP
DONE: POP // c
      BIPUSH 97
      IADD
      OUT // prints 'h' the letter with index 7 (length of collatz sequence)
```

```
      HALT
.end-main
```

As an example which also uses local variables and methods, consider the following C program, which computes the sum of n,n-1,…0 recursively.

```
        int sumem(int n) {
            if (n == 0) return 0;
            return n + sumem(n-1);
        }
        sumem(100)
```

The following JAS code is equivalent to the above C code (IJVM file for web implementations)

```
.constant
    objref  0xCAFE  // may be any value.  Needed by invokevirtual.
    COUNT 100
    .end-constant

.main

    LDC_W objref
    LDC_W COUNT
    INVOKEVIRTUAL sumem
    DUP
    IAND
    HALT
.end-main

.method sumem(var)
    ILOAD var
    IFEQ ZERO
    ILOAD var
    LDC_W objref
    ILOAD var
    BIPUSH 0x1
    ISUB
    INVOKEVIRTUAL sumem
    IADD
    IRETURN
    ZERO:
        BIPUSH 0x0
```

```
        IRETURN
.end-method
```

If you can understand these examples, and have done the installation and read the introduction to C, you are in good shape to start on task 1.

The files directory in the skeleton contains all the IJVM file we test with, you can always load them in the web-based implementations to see the correct behavior.

1 - Binaries, dreaded Binaries!

> ## What to implement
>
> ### Functions
>
> - `init_ijvm`
> - `destroy_ijvm`
> - `get_text`
> - `get_text_size`
> - `get_constant`
>
> In order to pass **test1**, correctly implement all of the functions listed above.

Introduction

In this chapter, you will be tasked with parsing an IJVM binary file. This means that you will read in an **.ijvm** binary file and place the IJVM program in memory in a way such that you can execute the program in the later modules.

IJVM binaries, like other file formats, consist of bytes, arranged in a meaningful way. The IJVM format is organized as follows:

```
        number of bytes | meaning

        4    BYTES  |  MAGIC NUMBER

        4    BYTES  |  CONSTANT POOL ORIGIN
        4    BYTES  |  CONSTANT POOL SIZE
        SIZE BYTES  |  CONSTANT POOL DATA

        4    BYTES  |  TEXT ORIGIN
        4    BYTES  |  TEXT SIZE
        SIZE BYTES  |  TEXT DATA
```

IJVM binaries consist of a 4-byte **magic number** followed by 2 blocks, namely the **constant pool** block and the **text** block. The constant pool block contains the constants and the text block contains the executable code. Each block starts with a 4-byte **origin**, which can safely be ignored, followed by another 4-byte **size** signifying the number of bytes the data consists of. The rest of the block contains the actual data.

It is your task in this module to: * Check the magic number (`0x1DEADFAD`) * Read in the constant pool and ensure that the method `get_constant` can return any constant * Read in the text data (program code) and ensure that the method `get_text` returns the correct byte array. This is tested in `tests/test1.c`

Note: the 4-byte origin is useless in this project, and is a leftover from the original IJVM file format where it indicated where the a block should be loaded in memory.

Hint: both blocks have the same basic layout, so creating a reusable function that parses a block and stores its data in a buffer will make your code more readable.

For a more intuitive view of the IJVM binary file format, see our interactive binary explorer below.

!INCLUDE "../binaryexplorer/binexplorer.html"

Reading the file

Please ensure you have read the section on reading files in the C introduction.

To read in the file, make calls to `fread` to read the parts of the binary, the magic number, the constant pool origin, and so forth until you have read and parsed the entire binary. The example below illustrates reading an integer from an IJVM binary:

```
        uint8_t numbuf[4];

        fread(numbuf, sizeof(uint8_t), 4, fp);
        uint32_t number = read_uint32(numbuf);
```

To read a ***signed*** 16-bit or 32-bit number, simply read in an 16-bit or 32-bit unsigned number, and then cast it to a signed number of the same size.

When you are reading the constants, note that the number of constants is the constant pool size divided by 4, as each constant is a word (4 bytes).

Endianness

When reading any data type larger than 1 byte in size from a file, the value that is obtained depends on the **endianness** of the system: the order bytes are stored in. Bytes can be stored in two different ways: big-endian and little-endian.

In **big-endian**, the bytes are stored from most significant (big-end) to least significant. This reflects the way numbers are written on paper: we start with the most significant digit. In **little-endian**, the bytes are stored in the opposite order, from least significant (little end) to most significant; the bytes are stored backwards. Suppose we have the number `0xFEE1600D`, where the most significant byte is 1D (numbers prefixed with `0x` are in hexadecimal). The order of the bytes is then:

```
// Little-endian
0x0D,0x60,0xE1,0xFE

// Big-endian
0xFE,0xE1,0x60,0x0D
```

The following program illustrates the difference between little and big endian, and you can compile it to know the endianness of your machine. If you understand this program and why it prints what it prints, you are in good shape. If parts of it are unclear, refer back to the introduction to C.

```
#include <stdint.h>
#include <stdio.h>

int main(int argc, char** argv){
  uint8_t bytes[] = {0x0D,0x60,0xE1,0xFE};
  // bytes[0] is now 0x0D, bytes[1] is 0x60 etc.
  // Cast uint8_t pointer to a uint32_t pointer
  uint32_t* wordpoint = (uint32_t *)bytes;
  // interpret above bytes as a word by dereferencing pointer
  uint32_t word = *wordpoint;
  int reversed = 0xFEE1600D; // hex number where FE is the most significant byte
  int straight = 0x0D60E1FE; // hex number where 0D is the most significant byte
  if(word == reversed) {
    printf("This is a little endian machine\n");
  } else if (word == straight) {
    printf("This is a big endian machine\n");
  } else {
    printf("This machine defies any logic\n");
  }
}
```

Ensuring the right endianness

The x86-64 or ARM architecture that your system is probably based on uses little-endian integers, while the IJVM binary file format uses big-endian integers. As a result, when you read a word from file, you will get the bytes in the wrong order: you will for example read in `0x0D60E1FE`, while `0xFEE1600D` was the actual number.

You have to ensure the right endianness of any word or short you read from the `ijvm` file. This means: * The magic number * The size of the constant pool * Each number in the constant pool * The size of the program text

The actual program text is interpreted as bytes, not words or shorts, and hence there is no problem with endianness. Some instructions (for example **GOTO**) have an argument that is a short. When parsing this short from the program text in in module 3, you need to ensure the right endianness of that short.

To ensure the right endianness of words and shorts, you can either:

- Interpret a series of bytes as a word (or short) and then swap the byte order.
- Convert a series of bytes to a word (or short) byte for byte, interpreting the first byte as most significant.

Swapping the byte order

If you read in words from a file, then the bytes are in the wrong order.

To reverse the order, i.e. to convert an integer from little-endian to big-endian, use the ***swap*** functions declared in `include/util.h`, which are implemented in `src/util.c`. The definitions of these are as follows:

```
        uint32_t swap_uint32(uint32_t num)
        {
            return ((num >> 24) & 0xff) | ((num << 8) & 0xff0000) | ((num >> 8) & 0xff00) |
((num << 24) & 0xff000000);
        }
```

There is also a version for a short (16 bits):

```
        uint32_t swap_uint16(uint16_t num)
        {
            return ((num >> 8) & 0xff) | ((num << 8) & 0xff00);
        }
```

Please refer to this discussion thread for an explanation of the function above.

Converting bytes into an integer

It is also possible to convert bytes into an integer use the ***read*** functions declared in `include/util.h`, which are implemented in `src/util.c`. The definitions of these are:

```
        uint32_t read_uint32(uint8_t* buf) {
          return (buf[0] << 24) | (buf[1] << 16) | (buf[2] << 8) | buf[3];
        }


        uint16_t read_uint16(uint8_t* buf) {
          return (buf[0] << 8) | buf[1];
        }
```

The example below breaks down the above methods:

```
int byte_1, byte_2, byte_3, byte_4;

byte_1 = 0xFE; // 0x000000FE
byte_2 = 0xE1; // 0x000000E1
byte_3 = 0x60; // 0x00000060
byte_4 = 0x0D; // 0x0000000D

// shifting 0x0000001D left by 24 bits (or 3 bytes) results in 0xFE000000
byte_1 = 0xFE << 24;
// shifting 0x000000EA left by 16 bits (or 2 bytes) results in 0x00E10000
byte_2 = 0xE1 << 16;
// shifting 0x000000DF left by  8 bits (or 1 bytes) results in 0x00006000
byte_3 = 0x60 << 8;
// shifting 0x000000AD left by  0 bits (or 0 bytes) results in 0x0000000D
byte_4 = 0x0D;

int number = byte_1 | byte_2 | byte_3 | byte_4; // 0xFEE1600D
```

Note that shift left `(l << s)` is defined as an operation which (efficiently) multiplies `l` by `2^s` (irrespective of endianness), so this operations above state “interpret the first byte as the most significant byte, the second byte as the second most significant, etc.”.

Hence, converting bytes to an integer this way will also work correctly on a big endian machine (see here for a blog advocating this method). This is not true for the method of interpreting the bytes as words and then switching the byte order which is explained above. However, we will not run your program on a big endian machine, and your program does not have to work on one.

Testing

To complete this task, you have to pass **test1**. To run this test, run `make run_test1`. We check if you read in the file correctly, by calling your `init_ijvm` method with one of the two `ijvm` files from `files/task1/` and then check if the text and constants have been read in correctly by calling your `get_text`, `get_text_size` and `get_constant` methods. You can view the source code for the test in `tests/test1binary.c` and the JAS files (which are compiled to IJVM files) in `files/task1/`.

Hints

- Make sure you have read the introduction to C.
- Do **not** program in **main.c**, put all of your code in **ijvm.c** instead.
- Do **not** compile your code manually, refer to the manual instead.
- Get familiar with **ijvm.h**.
- Add variables to your virtual machine by defining them in the struct in **ijvm_struct.h**

- Start with implementing the `init_ijvm` function, as the rest of the functions depend on information obtained as a result of parsing the binary.
- It is a good idea to use `dprintf` (explained in `util.h`) to debug your code.
- `malloc` returns a block of memory can already contain data (from the previous usage of the memory). Make sure you initialize all your variables. Our tests ensure that all memory returned by `malloc` is non-zero, to catch use of uninitialized value bugs early.

2 - Stack up!

> ## What to implement
>
> ### Functions
>
> - `get_program_counter`
> - `tos`
> - `step`
> - `finished`
>
> ### Instructions
>
> - `OP_BIPUSH`
> - `OP_DUP`
> - `OP_IADD`
> - `OP_IAND`
> - `OP_IOR`
> - `OP_ISUB`
> - `OP_NOP`
> - `OP_POP`
> - `OP_SWAP`
> - `OP_ERR`
> - `OP_HALT`
> - `OP_OUT`
> - `OP_IN`
>
> In order to pass **test2**, correctly implement all of the functions and instructions listed above.

Introduction

In this chapter, you will implement the basic stack operations of the IJVM, such as popping two words from the stack, adding them and then pushing the result (`OP_IADD`).

All IJVM stack operations operate on words, so the implemented stack should also operate on words.

The program counter

The program counter is an index in the text data that points to the current instruction. It should be initialised to `0` at the start of your program and incremented accordingly with each executed instruction.

The suggested approach

Start with implementing the stack, as all the functions and instructions depend on it. The simplest and fastest way to implement the stack is to simply allocate an array for the data on the stack and keep track of its maximum capacity and current size. As a reminder: a stack is a LIFO (last in, first out) data structure with three main principal operations: * `push`: add an element to the top of the stack * `pop`: remove the element on the top of the stack and return it * `top`: return the element on the top of the stack

Once you've implemented the stack, shift your focus to the `step` function. The `step` function executes the current instruction (i.e., the instruction pointed to by the program counter). Consequently, your `step` function must be able to handle all the instructions required in order to pass `test2`.

Each consecutive chapter will introduce a new set of instructions for you to implement, so your emulator will eventually be able to handle the entire IJVM instruction set. It is a good idea to use a switch statement to determine how to handle the current instruction.

The instructions for task 2 are:

OpCode	Instruction	Args	Description
0x10	BIPUSH	byte	Push a (signed) byte onto the stack. **Note:** Cast the unsigned byte to a signed char first, then to int—otherwise you get 0–255 instead of –128–127.
0x59	DUP	N/A	Duplicate top of stack: Push a copy of the top word onto the stack
0x60	IADD	N/A	Pop two words from the stack and push their sum
0x7E	IAND	N/A	Pop two words from the stack and push their bitwise AND

OpCode	Instruction	Args	Description
0xB0	IOR	N/A	Pop two words from the stack and push their bitwise OR
0x64	ISUB	N/A	Pop two words from the stack and push their difference **Note:** subtract the top word from the second to top word
0x00	NOP	N/A	Do nothing
0x57	POP	N/A	Pop a word from the stack
0x5F	SWAP	N/A	Swap the two top words on the stack
0xFE	ERR	N/A	Print an error message to the machine output and halt the emulator
0xFF	HALT	N/A	Halt the emulator
0xFC	IN	N/A	Read a character (byte) from the machine input and push it onto the stack. ***If no character is available, push 0 instead.***
0xFD	OUT	N/A	Pop a word from the stack and print it to the machine output as a character

Hints

- Please refer to the **ijvm.h** header file for a description of each function in **ijvm.c**
- Since you are going to use a stack, it makes sense to implement a stack data type (i.e., a struct).
- Remember to add some basic error handling to your code (i.e., assert that the stack is not empty when `pop` is called). Adding these kinds of checks will help you a lot down the line.

- The size of the stack should not be fixed, it should grow when the stack is full. Some tests check this by needing a very large stack. You can use `realloc` for example.
- To implement the `IN` instruction you have to read a byte from the `in` file description, which you can do with `fgetc(in)`. If no character is avaible, `fgetc` will return `EOF`. Similarly, to write a byte `value` to `out`, use `fprintf(out, "%c", value)`.

3 - Controlling the Flow: the GOTO solution!

> ## What to implement
>
> Update your `step` function.
>
> ### Instructions
>
> - `OP_GOTO`
> - `OP_IFEQ`
> - `OP_IFLT`
> - `OP_IF_ICMPEQ`
>
> In order to pass **test3**, correctly implement all of the functions and instructions listed above.

Introduction

In this chapter, you will be tasked with implementing basic branching. The IJVM instruction set contains four instructions that control the flow of the program: **GOTO**, **IFEQ**, **IFLT** and **IF_ICMPEQ**. The idea behind the **GOTO** instruction is simple: add an offset to the program counter, and continue executing the program from that address. Other instructions such as **IFEQ** do the same, but only branch if a certain condition is met.

To illustrate how **GOTO** should behave, consider the following example program which prints **13** (and skips printing ***2***):

```
        .main
        L1:
          BIPUSH 0x31 // Push  '1'
          OUT         // Print '1'
          GOTO L3     // Jump to L3

        L2:
          BIPUSH 0x32 // Push  '2'
          OUT         // Print '2'

        L3:
          BIPUSH 0x33 // Push  '3'
```

```
        OUT         // Print '3'
        HALT

    .end-main
```

In contrast to the instructions you've implemented so far, the **GOTO**, **IFEQ**, **IFLT** and **IF_ICMPEQ** instructions take a **signed short** as an argument. The figure below illustrates the layout of different IJVM instruction formats:

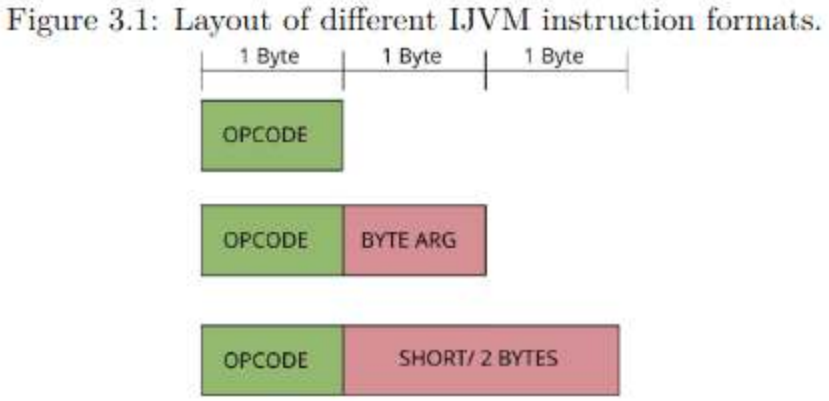


Figure 3.1: Layout of different IJVM instruction formats.

As the argument of these instructions is a signed ***short***, you have to ensure the right endianness as described in Module 1.

The suggested approach

Start with implementing the **GOTO** instruction, then move to the other three.

OpCode	Instruction	Args	Description
0xA7	GOTO	short	Increment the program counter by a (signed) short
0x99	IFEQ	short	Pop a word from the stack and branch if it equals zero
0x9B	IFLT	short	Pop a word from the stack and branch if it is less than zero
0x9F	IF_ICMPEQ	short	Pop two words from the stack and branch if they are equal

Hints

- Keep in mind that the offset is calculated based on the program counter value at the beginning of the branching instruction, so you might obtain an incorrect value if you directly increment your program counter to the point at the argument and then do the jump.
- Some instructions, such as the branching instructions, have arguments that are signed, while other instructions have an unsigned argument. In general, instructions that take an index as argument have an unsigned argument, while other instructions have signed arguments.

4 - Local variables: Artisan and Organic!

> ## What to implement
>
> ### Functions
>
> - `get_local_variable`
>
> ### Operations
>
> - `OP_LDC_W`
> - `OP_IINC`
> - `OP_ILOAD`
> - `OP_ISTORE`
> - `OP_WIDE`
>
> In order to pass **test4**, correctly implement all of the functions and instructions listed above.

Introduction

In this chapter, you will be tasked with implementing loading constants onto the stack and handling local variables.

The constant pool

The constant pool is the location in memory which contains read-only constants. These constants are loaded into the constant pool at load-time, and are never changed thereafter. In task 1, you already loaded in the constants from file (and ensured the right endianness). The only thing you have to do now is push the desired constant on the stack when encountering and **LDC_W** instructions by using `get_constant` you implemented earlier.

Local variables

Next to the stack, data can also be in stored local variables. Each local variable contains a word of data, which can be loaded on the stack using **ILOAD**. The **ISTORE** instruction pops the topmost stack word to the given local variable .

Consider a small example which uses **ILOAD** and **ISTORE**:

```
.main
    .var
    a
    b
    .end-var
    BIPUSH 0x1 // stack: 1
    ISTORE a   // stack:
    BIPUSH 0x2 // stack: 2
    ILOAD a    // stack: 2, 1
    IADD       // stack: 3
    ISTORE b   // stack:
    ILOAD b    // stack: 3
    ILOAD b    // stack: 3, 3
    HALT
```

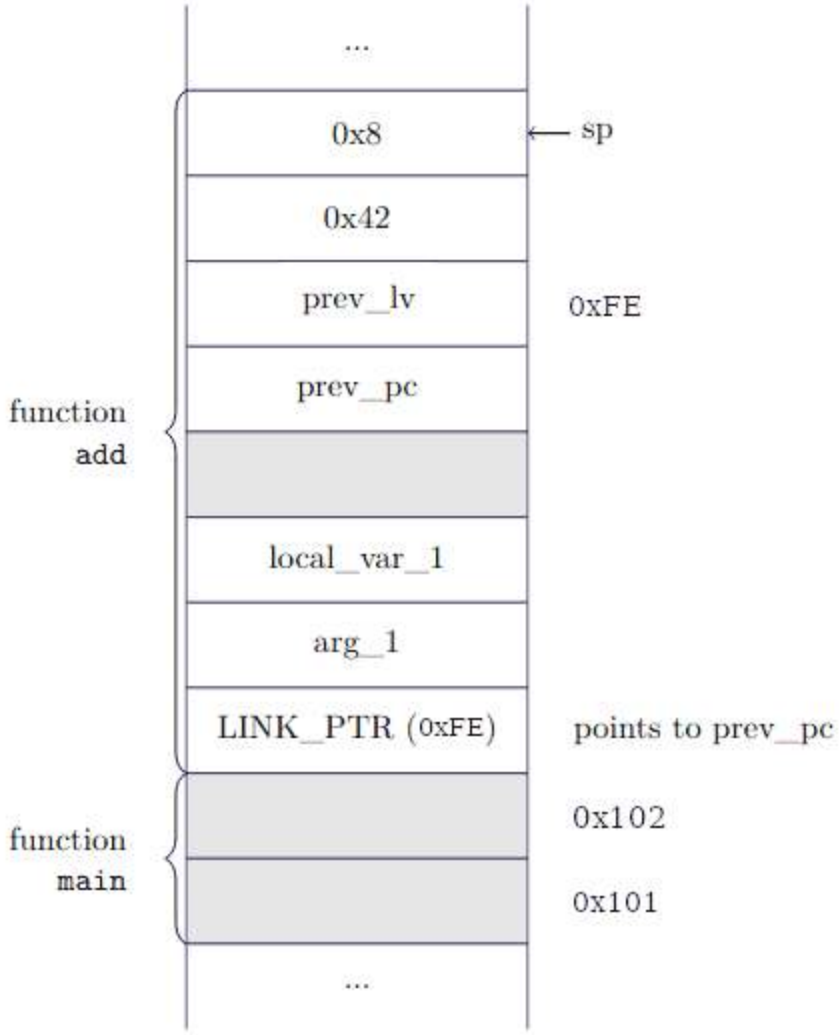
In this example, main has two variables, **a** and **b**. Above, these variables are referred to by name, but in the ijvm file these are replaced with indices, starting from 0. In this example `a -> 0` and `b -> 1`. Hence, `0x15(ILOAD) 0x01`, is the translation of `ILOAD a` above.

As we will see in the next task, the ijvm binary lists for each method how many local variables there are, except for main. You can safely assume that the `main` method contains a maximum of **256** local variables.

Frames

The main challenge with local variables is to store them in a way that also works once you also implement methods in the next task. Local variables are stored in a structure that contains all information for the current method: ***the local frame***.

All IJVM methods, including `main`, are assigned their own local frame. Each local frame contains the arguments, local variables and return data: the data that is needed to return from a method. Consequently, the IJVM stack is built up of (local) frames, one for each method invoked. Frames are typically stored on the stack, and the layout is typically as follows:



IJVM Memory Model

For methods other than the first method, main, the bottom of the frame contains a link pointer which points to the location at which the previous program counter is stored (this so that you can find it regardless of how many elements are stored on the stack above it). For the main method this link pointer is omitted, as there is no previous program counter. To keep track of where the current frame starts, we maintain a frame pointer in a variable called **lv** (for local variable), which always points to the start of the frame. In the diagram, **lv** would point to the cell holding **LINK_PTR**. The local variables of the current frame can always be found at their offset from **lv**. The current frame is area between **lv** and the top of the stack **sp** (including both words pointed to).

Above the link pointer, you can find the method arguments and local variables. Above the local variables, you can find the return data: the previous frame pointer `prev_lv` and the previous program counter **prev_pc**. The values stored at these locations are restored appropriately when a method returns.

Arguments are handled in the same way as local variables, there is no distinction between the two other than that arguments are initialized by values pushed before the method call. Both are handled in the same way via **ILOAD** and **ISTORE**. The assembler gives every argument and local variable a unique label, which happens to be their index in the local frame. For example, if a method has a single argument `x` and two variables `a` and `b`, the assignment would be `x -> 1`, `a -> 2` and `b -> 3`. The index is counted from local variable pointer `lv`, so the location of an argument with index `i` is `lv + i` (if the stack grows upwards). Note that for methods other than main, the local variable/argument at index 0 is the link pointer itself, which before method invocation contains the `OBJREF` (see next chapter).

In the main method, there is no `OBJREF`, so the local variables start at 0 instead of 1. In this diagram, we assume there is space for 256 local variables (`0x100` in hex) and that 2 items were pushed on the stack before calling the method, hence the last value of the frame of main is at `0x102`. There hence is no space for the link pointer, but this is not a problem as there is no caller to return to. You can assume that `IRETURN` is never called in the main method, but if you want to be complete you can detect if this happens (by for example checking if lv points to the start of the stack), and then halt the program.

The suggested approach

If your method of storing and loading local variables does not also work in combination with the method which are introduced in the next chapter, you may have to redesign you implementation later. Hence, make sure you first also read the next chapter on methods.

Then, start with implementing the **LDC_W** instruction.

Afterward, shift your focus to local variables. The simplest option is to store the local variables exactly as described above. If you find this hard or illogical, you can also try another method: local frames can be implemented separately from the operand stack. You could, for example, implement them using a separate stack or a linked-list.

Once you have implemented **IINC**, **ILOAD** and **ISTORE**, implement **WIDE**. The **WIDE** instruction is a prefix instruction that indicates that the index argument of the subsequent instruction is represented with a short, as opposed to a byte, and is thus 2 bytes long. Only the **IINC**, **ILOAD** and **ISTORE** instructions can be prefixed with **WIDE**. The instruction **WIDE** is not considered a step itself, so **WIDE ISTORE 0x0000** is one step of the `step` function, not two.

OpCode	Instruction	Args	Description
0x13	LDC_W	short	Push the constant with index short from the constant pool onto the stack
0x15	ILOAD	byte	Push the local variable with (unsigned) index byte onto the stack
0x36	ISTORE	byte	Pop a word from the stack and store it in the local variable with (unsigned) index byte
0x84	IINC	byte byte	Increment a local variable by a value. The first byte is the (unsigned) index of the local variable. The second byte is the (signed) value.

OpCode	Instruction	Args	Description
0xC4	WIDE	N/A	Prefix instruction; the next instruction has a 16-bit (unsigned) index. This can be prefixed to ILOAD, ISTORE and IINC. This only changes the size of the index to 16 bits, the constant of IINC remains 8 bits.

Hints

- The main challenge of this task lies in creating a solid foundation for methods, which will be covered in the next chapter. A bad design may come back to haunt you, so think ahead.
- Our tests assume that the `step` function executes the whole **WIDE**-prefixed instruction in one step. In other words, the **WIDE** instruction and the subsequent instruction are treated as a single instruction.

5 - Call yourself a method!

> ### What to implement
>
> #### Operations
>
> - `OP_INVOKEVIRTUAL`
> - `OP_IRETURN`
>
> In order to pass **test5**, correctly implement all of the instructions listed above.
>
> **Note:** ideally, if you pass all of the basic tests, you should also pass most, if not all, of the advanced tests.

Please make sure you (re)read the chapter on the IJVM from the book ***Structured Computer Organization*** for a more in-depth explanation of IJVM method invocation.

Introduction

In this chapter, you will be tasked with implementing methods. Method invocation in the IJVM can be a bit tricky, as it has some strange behavior that is left over from the JVM (Java Virtual Machine). For example, when invoking a method, the

caller has to push an `OBJREF` (object reference) onto the stack as the first parameter. Since the IJVM does not support different objects, pushing this reference has no meaningful functionality. Nevertheless, it's still there just to keep (some kind of) compatibility with Java. This means all methods have a (useless) additional first argument: the object ref. For example, a function `add(x,y)` would get arguments `object ref, x, y`.

In a nutshell there are four things you need to do to invoke and return from a method:

1. Find the method area
2. Get the arguments
3. Set up the local frame
4. Move the program counter
5. Return to the previous frame

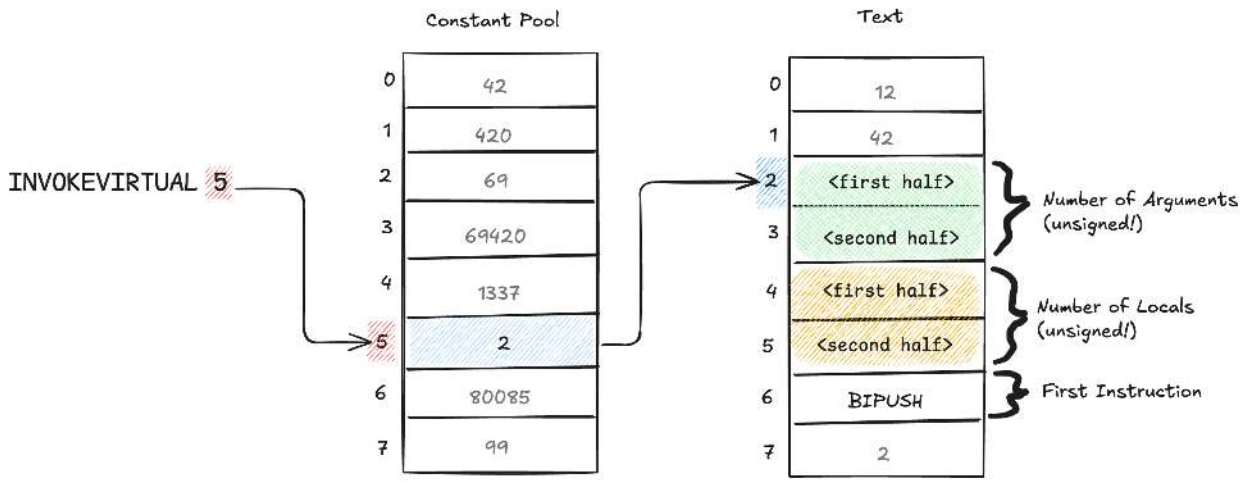
For the remainder of this chapter we will be using the example of calling `INVOKEVIRTUAL 5`, just to make the process easier to visualize.

Finding the method area

To invoke a method, the caller first pushes method arguments (including the object reference) onto the stack and executes the **INVOKEVIRTUAL** instruction. This instruction itself has one argument, which is the index of an element in the constant pool. The value of this constant is the address of the start of a section of the program text called the **method area**, which stores the number of arguments and the number of local variables of the method and its method text.

The method area first contains two shorts, the first one signifying **the number of arguments** the method expects (including `OBJREF`), and the second one being **the number of local variables**. **Note:** both of these numbers are **unsigned**. The fifth byte in the method area is the actual first instruction to be executed, so adjust your program counter accordingly!

Confused? That's okay! Below is an example of calling `INVOKEVIRTUAL 5` and how we find the method area.



Finding the method area

Here is what is happening: 1. `INVOKEVIRTUAL 5` was executed. This means: Go to the **constant pool** at index 5 2. There we find the number 2. This means: Go to index 2 in the **text** 3. There the **method area** starts. 4. The **first 2 bytes read together** are the number of arguments 5. The **following 2 bytes read together** are the number of locals 6. The **5th byte** is the first instruction

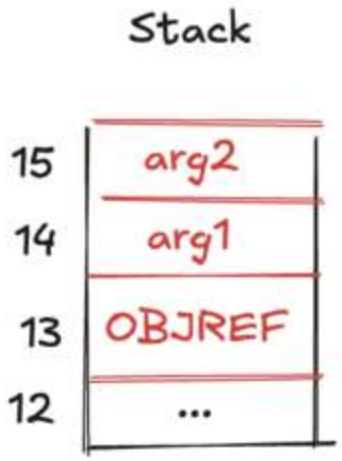
You might wonder why the IJVM employs this roundabout behavior, where we get a pointer from an index in the constant pool, instead of directly getting the pointer. This behavior is left over from the JVM, where it makes ***dynamic linking*** possible. On the JVM, if a method is not from the present program but from an external library, then the constant pool initially contains a symbolic reference of the external method, i.e., the name of the library and the method, instead of the actual address of the method (because there is no actual address yet, the library still has to be loaded). When such a symbolic reference is encountered, the library is loaded into memory, and the symbolic reference in the constant pool is changed to a pointer to the method in memory. This indirection makes this possible.

Getting the arguments

Alright, so you've now found where the method starts, and you have its number of locals and number of arguments. But where are the actual arguments, you might ask? They are already on the stack! The caller pushed them before calling INVOKEVIRTUAL, in order. Note that the `OBJREF` that we talked about earlier will always be the first argument, and that **`OBJREF` is included in the argument count**.

The locals, as you might remember, are slots for variables that the method will use during its execution. We now know how many it will need, so what we need to do is create space for the method to use these. In the next step we'll cover the most common way to solve this, but remember that any way is okay as long as it works.

Here is what the stack would look like in our example:



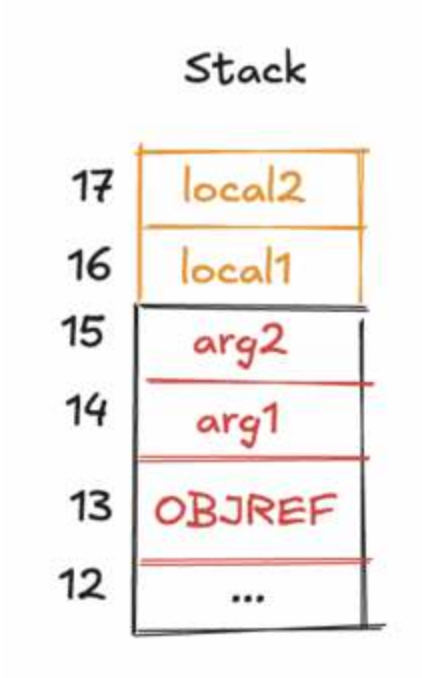
stack at the start

Setting up a local frame

Alright, recap: remember that we called `INVOKEVIRTUAL 5`, with 3 arguments and 2 locals, and that the caller already pushed the arguments to the stack. Note that this is just an example; the indices here could be anything.

Remember how we said the locals and args are expected to be next to each other? Well, that's easily solved here. We can simply add space for the locals on top of the stack, like so:

Note: Arguments and locals are always indexed together! What does this mean? In the below example the index of `local1` would be `3`, because we always start indexing from `OBJREF` (the first argument).



stack with locals

Since we are moving the program counter when we call a method, we need a way to get the previous program counter back. Saving the old PC lets us restore it during IRETURN, so execution in the caller resumes exactly at the instruction following INVOKEVIRTUAL. We can solve this by also pushing the previous program counter to the stack like so:

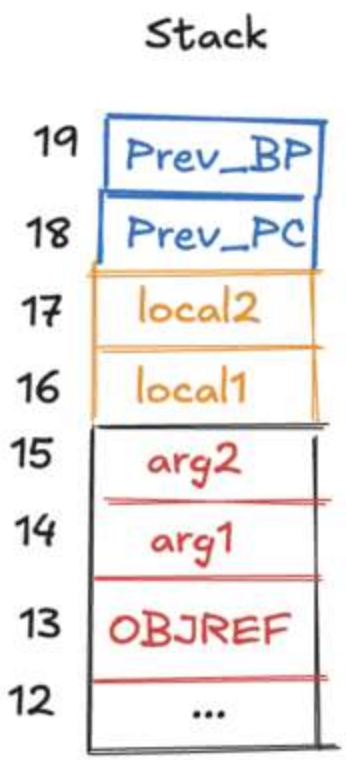
Note: Ensure that you are pushing the correct value. When the method is done executing we want to jump to the instruction that comes **after** the method, not the method itself.

stack with pc

Now you may ask yourself: “Well, sure we have what we need on the stack, but during execution how do we know where the current method’s stack frame begins and where it ends?”. That’s an excellent question, and this is where the **base pointer (BP)** comes in. (This was called **LV** (for local variables) in the previous chapter)

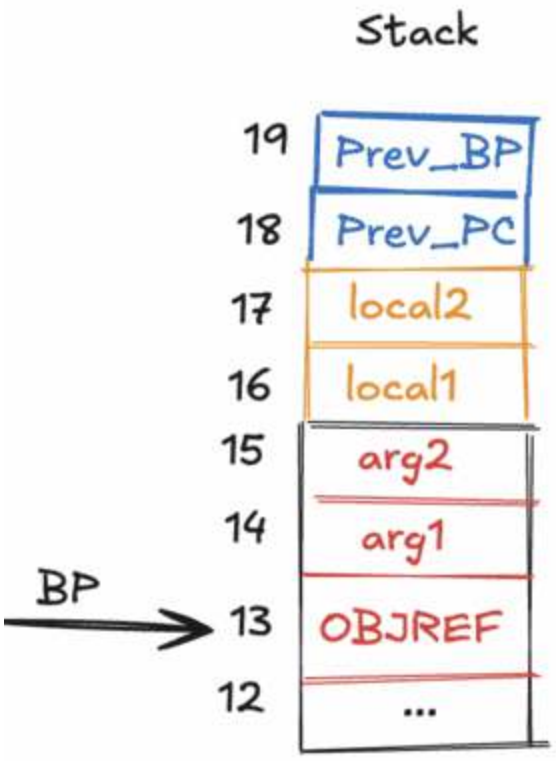
The base pointer is just a pointer to the bottom of the current frame. Since the `OBJREF` is always the first thing pushed, this is what our base pointer will point towards. Since we need to save our previous base pointer just like we needed to save our previous program counter, we first need to push it before we can point the current one to the bottom.

So in summary, first we push our previous base pointer:



stack with bp

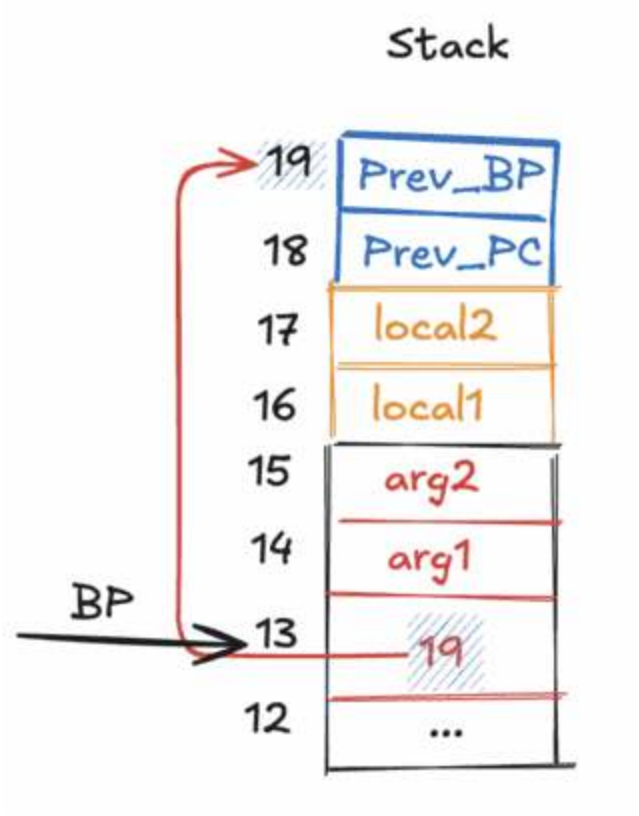
Then we make it point to the bottom of the current frame:



stack with new bp

Now you might also ask yourself: “Okay, so we know where the frame begins now. But how do we know where to find our `Prev_PC` and `Prev_BP`? A method can push whatever it wants to the stack, right?”. Correct (If you’re confused by this, don’t worry. It will be elaborated on in the section on returning).

There’s a trick we can perform here though. Remember how `OBJREF` is unused in our program? Well, since it’s useless, we can just overwrite it with whatever we want! So we can just overwrite it with the index of the top of our frame:



stack at the end

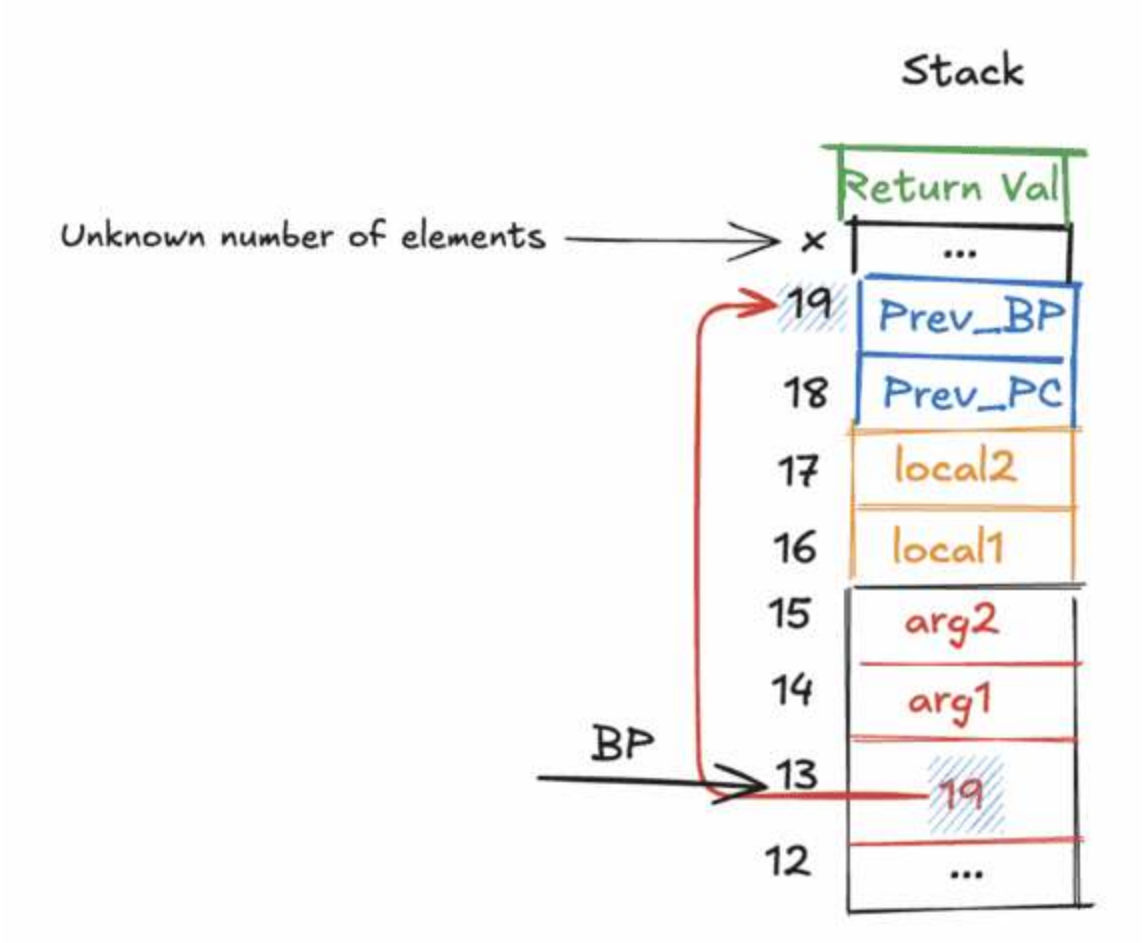
Moving the program counter

Now that everything is in place you are safe to move the program counter to the first instruction of the method! After this the method will start executing, and we will know it's done when it calls `IRETURN`. Which leads us right into…

Returning

Alright, so the method has now informed us that it is done running by calling `IRETURN`! Now there are a few things we need to do. First, we need to get the return value, then we need to restore our old base pointer and program counter.

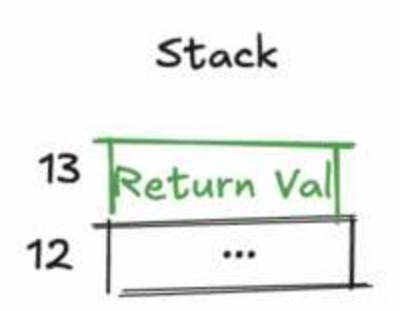
Finding the return value is easy! When a method returns, the top of the stack will contain the return value. **However:** Between the top of the stack and `Prev_BP` there could be **any number of elements!** So we can't just keep popping and expect to get our `Prev_BP` and `Prev_PC` back!



before returning

Alright, so we can't find `Prev_BP` and `Prev_PC` by just popping what's below the return value. How do we find them then? Well, let's look at our base pointer! It currently points to `index 13`, which is the bottom of the frame. At that index we have the number `19`, which is the **index of `Prev_BP`**! So using that index, we can now find `Prev_BP` and `Prev_PC` and restore them!

Once you've safely restored all of these values, all we need to do is remove the entire frame, then make sure that the return value is at the top of the stack. That's it! Here's what the stack should look like when you are done:



after returning

Overview

A complete example of how all of this works, is demonstrated in [this interactive webpage by past TA Zain Munir](#).

Below are the images from the book. In these images what we call `BP` (base pointer) above is called the `LV` (local variables).

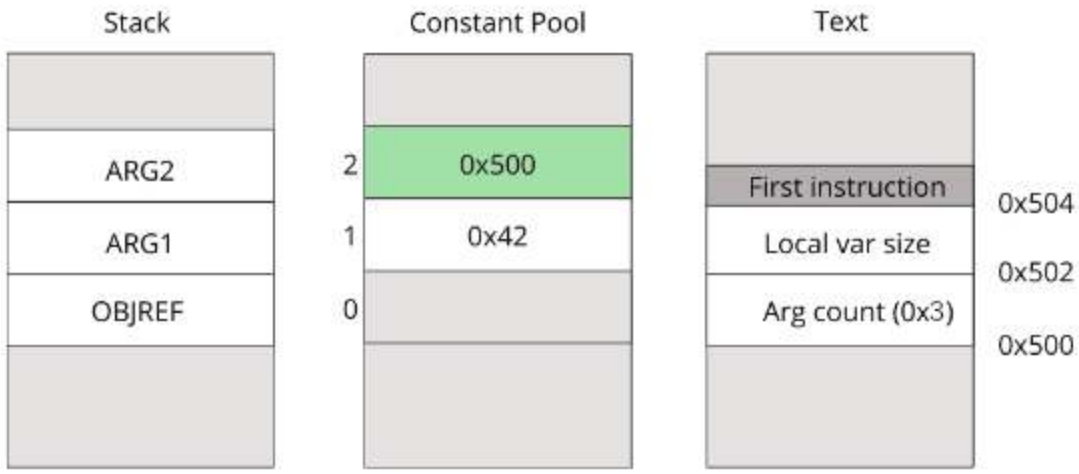


figure 5.1

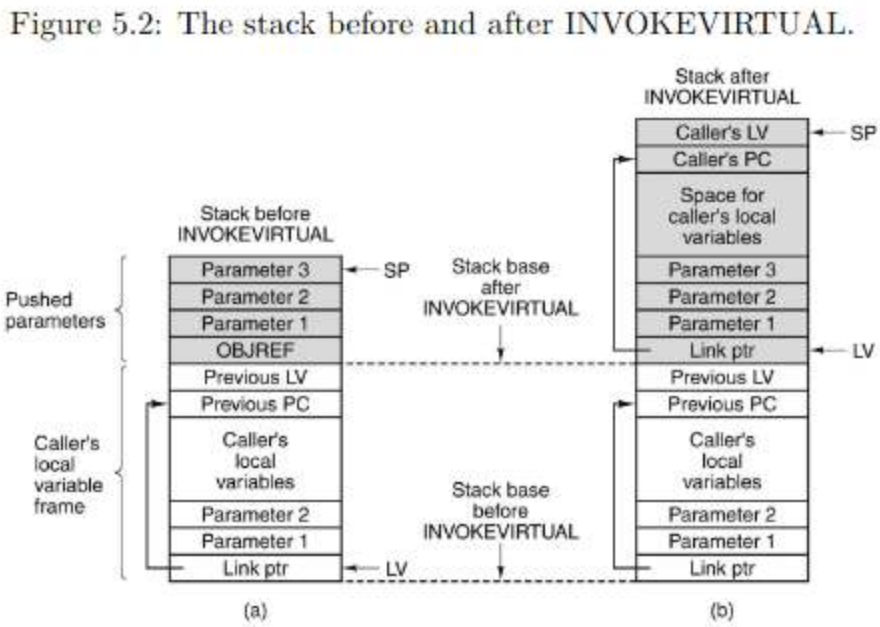


figure 5.2

The suggested approach

Before you start implementing any instructions, make sure you keep track of the current frame using a base pointer **bp** and are accessing the local variables as an offset from that.

Once you've made sure you keep track of the current frame, start implementing the **INVOKEVIRTUAL** and **IRETURN** instructions.

OpCode	Instruction	Args	Description
0xB6	INVOKEVIRTUAL	short	Invoke a method
0xAC	IRETURN	N/A	Return from the current method

Done? Did you pass all basic tests and at least 5 of the advanced tests? Hurray! Your program is sufficient to pass the course.

6 - Even more stuff?! Because why not?

You need to pass all of the basic tests and at least 5 out of the 8 advanced tests to be eligible for bonus points.

Please list which additional features you implemented in `bonus.md` in the root directory of your project (which is included when using `make zip`). You can format your file using markdown

---

If you want to compile your own `jas` files to `ijvm` binaries, you can use goJASM. You can install goJASM with `make tools`, which will install the `goJASM` binary in the `tools` directory.

Introduction

You can get **at most** 3.5 points by implementing additional features. Keep in mind that some features are harder than others, as indicated by the number of points awarded for them.

Some of these additional features come with their own automated tests, while others don't. If a feature doesn't come with any automated tests, then testing its functionality yourself is part of the assignment.

Feature	Base	Additional	Total
Tail call	0.5	N/A	0.5
Heap memory	0.5	1.5	2
Networking	0.5	0.5	1
Snapshots	1	N/A	1
Hardening	1	N/A	1

Feature	Base	Additional	Total
GUI	1	N/A	1
Debugger	1.5	1.5	3

Tail call

Opcode	Instruction	Args	Description
0xCB	TAILCALL	short	Invoke a method and then return, in an optimized way.

Extend your emulator with tail calls by implementing the **TAILCALL** instruction, which takes the same argument as **INVOKEVIRTUAL** to indicate the method being called.

The **TAILCALL** instruction is equivalent to an **INVOKEVIRTUAL** instruction followed by an **IRETURN** instruction, with the only difference being that the caller's stack frame is cleaned up before executing the method, instead of after **IRETURN**.

You can test your implementation using **testbonustail**. To use this test, implement the `int get_call_stack_size(void)` line in `ijvm.c` (description in ijvm.h). Build and run the test with `make run_testbonustail`. Note that passing the test is not enough for getting the points, a TA will ensure that you indeed clean up the caller's stack frame before executing the method instead of after.

Heap memory

All information in the IJVM is stored on the operand stack, however it is sometimes necessary to have persistent memory between method calls. Extend your interpreter with a heap by implementing the three instructions below.

Opcode	Instruction	Args	Description
0xD1	NEWARRAY	count	Create a new array on the heap. The count is popped from the stack and replaced by an array reference that can be used to refer to the newly created array. All values in the array should initialized to zero.
0xD2	IALOAD	index, arrayref	Push the value stored at location index in the array referenced by arrayref onto the stack

Opcode	Instruction	Args	Description
0xD3	IASTORE	value, index, arrayref	Store value at location index in the array referenced by arrayref

Note: the arguments of these instructions are placed on the stack in the order in which they appear (i.e., the last argument is at the top of the stack when the instruction is executed).

You should also implement **bounds checking** for the instructions. For example, if you use an index for the **IALOAD** instruction that goes beyond the length of the referenced array, you should print an error message and halt your interpreter.

You can test your implementation using **testbonusheap**. Use `make run_testbonusheap` to build and run the test.

Garbage collection

Opcode	Instruction	Args	Description
0xD4	GC	N/A	Free all unused heap arrays

Extend your heap with garbage collection by freeing all **unused** heap arrays periodically, or when the program encounters the **GC** instruction. A heap array is unused if no reference to the heap array can be found anywhere in the interpreter, including other heap arrays. Your garbage collector should work with circular references: if heap array A references heap array B, and vice versa, but no other memory references array A or B, then both should be freed.

The easiest to implement is a ***conservative*** garbage collector, a garbage collector does not actively keep track of which values are references and which are not. If the garbage collector sees a value in memory that could be a reference (because it matches the address of some allocated memory), it assumes that it is a reference and refrains from freeing the memory in question, even if that value is actually an integer or some other kind of non-pointer data that just happens to have the same bit pattern. To make the chance of a false positive smaller, you can employ a strategy that ensures that array references are not small numbers (0,1,2 etc) as they are more commonly used. Conservative garbage collectors can even be used in situation which were not designed for garbage collection - for example there exists a conservative garbage collector for C which can also be used to find memory leaks.

You are free to choose how to implement the garbage collector. You could, for example, keep track of existing references to arrays or implement a simple mark-and-sweep mechanism.

We suggest implementing the following mark-and-sweep strategy: * ***Mark:*** On garbage collection traverse the entire stack (not only the frame of the current method, but also all other frames on the stack), including all local variables and arguments on the call stack, making sure to skip the link pointer, previous program counter and previous stack pointer. For any integer you encounter, if there is a corresponding heap array, mark that as used. Visit that array and do the same for all the values there. * ***Sweep:*** Free all heap arrays which were not marked as used in the mark phase. Note that this strategy also works for circular references: if heap array A references heap array B, and vice versa, but no other memory references array A or B, then both are freed after this procedure.

The garbage collection should print a message whenever it is triggered, and include the **GC** instruction that manually triggers garbage collection. You should also include some documentation in the `bonus.md` file on how you implemented your garbage collection.

You can test your implementation using **testbonusgarbage**. To use this test, implement `bool is_heap_freed(word_t reference)` in `ijvm.c` (description in ijvm.h). Use `make run_testbonusgarbage` to build and run the test. Note that passing the test is not enough for getting the points, a TA will also check your garbage collector.

For more of a challenge, you can also implement a ***precise*** garbage collector: a garbage collector which actively keeps track of which values are references and which are not - and hence cannot leak memory due to mistaking an integer for a reference. Each value must have a bit indicating wether it is a reference or not. As integers are 32-bit, there is no space within these 32 bits to store an extra bit. An easy, but wasteful solution is to store all values in 64 bits instead, using 32 bits for the value and 1 bit to indicate wether the value is a reference or not, and leave 31 bits unused (if you do this make sure you still use 32 bit arithmetic!).

A less wasteful solution is to store the bit externally, for example all local variables/arguments together can have a ***bit-map*** together: some words near the local variables where the first bit indicates if the first local variable is a reference, the second bit indicates if the second local variable is a reference and so on. You also need such a bit-map for the values on the stack. Note that in the current setup you cannot specify wether an array should hold references or integers, hence arrays can mix references and integers and hence you also need a bit-map of all values in array. In the real JVM instruction set, there are different instructions for creating an array of references and an array of primitive values, so all we then need is a single bit for the entire array. To mimic this behaviour, you can also choose to introduce three new instructions ***ANEWARRAY***, ***AIALOAD*** and ***AIASTORE***, which work exclusively for arrays of references and keep the old instructions exclusively for integers. Use `make run_testbonusprecisegarbage` to test your precise garbage collector. Use `make run_bonusaltgarbage` instead of `make runbonusgarbage` if you use ***ANEWARRAY***, ***AIALOAD*** and ***AIASTORE*** for reference arrays. (Thanks to Kevin Lam for these tests!)

Opcode	Instruction	Args	Description
0xBD	ANEWARRAY	count	Create a new reference array on the heap.
0x32	IALOAD	index, arrayref	Push the reference stored at location index in the array referenced by arrayref onto the stack
0x53	IASTORE	value, index, arrayref	Store reference at location index in the array referenced by arrayref

Scoring: The garbage collector can give you 1-1.5 points on top of your 0.5 points for implementing the heap. You get 1 point for a conservative garbage collector or a precise garbage collector where all values are stored in 64 bits and 1 bit is used to store whether something is a reference or not. You get 1.5 points for a precise garbage collector as described in the previous paragraph. In a precise garbage collector, you can choose to have a bitmap for arrays, or implement the extra instructions ***ANEWARRAY***, ***AIALOAD*** and ***AIASTORE***.

Networking

Opcode	Instruction	Args	Description
0xE1	NETBIND	port	Pop port from the stack and start listening for a network connection on it. Push a netref (a positive integer indicating the connection) on the stack upon success, otherwise push 0.
0xE2	NETCONNECT	host, port	Open a network connection on the specified host (ipv4 address encoded in one word) and port. Push a netref (a positive integer indicating the connection) on the stack upon success, otherwise push 0.
0xE3	NETIN	netref	Pop netref from the stack, receive a character from the network connection referenced by it and push the character onto the stack
0xE4	NETOUT	char, netref	Pop netref and a word from the stack and send that character to the network connection referenced by netref
0xE5	NETCLOSE	netref	Pop netref from the stack and close the network connection referenced by it

Note: the arguments of these instructions are placed on the stack in the order in which they appear (i.e., the last argument is at the top of the stack when the instruction is executed).

As seen in the course ***Computer Networks***, most modern applications depend on networked access to other devices. Extend your emulator to support **one** network connection. You are allowed to use the GNU C library (see **sys/socket.h** and **arpa/inet.h**) to implement the networking. See this article for an overview of important socket programming concepts and example code. Since in this assignment there is only one network connection, netref can always be 1.

There are currently no automated tests for this feature, however we have provided you with some IJVM binaries you can use to test your implementation. You can find these binaries in **files/bonus**. The provided binaries are one-sided: the server is provided by **test_netbind.ijvm** and as the client is provided by **test_netconnect.ijvm**.

Currently, the server and client do the same thing: recieve two bytes and send them back in reverse. You can hence not connect the server to the client. It is up to you to act as the corresponding counterpart (i.e., client or server) in both cases.

To achieve this, use a networking utility, such as netcat, or write a simple script in Python (or any other language of your choice) that handles the other end of the connection. You can also adjust and compile the server or client such that they can be connected to each other.

Multiple connections

Extend your emulator to support **multiple** network connections. Use the `netref` argument as a network connection identifier. Modify the **NETBIND** and **NETCONNECT** instructions to push a meaningfull `netref` onto the stack instead and modify the **NETIN** and **NETOUT** instructions to make use of the `netref` to distinguish between different network connections.

There are currently no automated tests for this feature, however it is sufficient to modify the provided **test_netbind.jas** and **test_netconnect.jas** files to support multiple connections. Keep in mind that the modified JAS files have to be recompiled using goJASM.

Snapshots

One benefit of virtual machines is that the machine is (surprise) virtualised. This allows for convenient features, such as migration and snapshots. Extend your emulator with snapshots by saving the state of your IJVM instance to a file when receiving a `SIGINT` signal (Ctrl + C). To handle `SIGINT`, you need to implement a **signal handler**.

One thing of note is that when you trigger your signal handler your program could have not yet fully completed a step, thus you need to account for this case by letting the current step fully complete and ***only then*** serializing the state such that, upon restoring, the machine executes with a valid intial state. **This must be implemented to receive the bonus point**

You need to serialise the state of your machine. Store all specifics like the program counter, local variables and other memory contents in the snapshot. It is important to note that you should not save `FILE` pointers when saving input and output files, as files can be temporary.

You then have to be able to start up a new IJVM instance (add a command-line option to your emulator) from the serialised file, continuing at the state where you saved the snapshot. It is up to you to decide the format of the serialised file.

There are currently no automated tests for this feature, however it is sufficient to be able to recover from a `SIGINT` when running **testadvanced6**.

Hint: Signal handlers return you to back to the place where you executed from with any modifications from the signal handler function applied to your program. To read up more on how to properly work with signal handlers checkout this resource

Hardening

Harden your emulator. Make sure your program does not crash due to memory errors (i.e., segmentation faults) even when given an invalid binary.

We do not expect you to catch all possible errors in your program, mainly due to limited time. Our standard policy is you should make a best-effort to check for errors.

An example of an easily exploitable part is the offsets used in the binary. If the index value of the **ISTORE** instruction is not checked, this could be used to divert the control flow of the program.

Make sure to check for overflows and underflows and check the sizes of values that you read. Make sure to use tools, such as fuzzers and sanitisers, to find bugs in your implementation. See this guide on fuzzing to get started.

Document the hardening process (in the README) and quantify the improvement (e.g., using code coverage). Also describe your development process (i.e., how you found the errors in your code, what inputs were able to crash your program, etc.) and how your changes might affect performance (if applicable).

Your program must compile with `make pedantic` and pass `testsanitizers`.

Requirements:

1. Set up a fuzzer, such as AFL, to find bugs and let it run for about an hour
2. Analyse how much of your code gets covered by our automated tests and or by the test cases generated by the fuzzer
3. Write a short report in a `bonus.md`` file on what you did to mitigate bugs and errors. The main challenge of this additional feature lies with analysing what the attack surface is, what you can guarantee about your implementation and how to mitigate possible exploitation.

Note: hardening your emulator is the most open-ended of all the additional features. You have a lot of freedom in how you approach this. Keep in mind that you might have to set up a bunch of different tools and learn some new concepts.

GUI (Graphical user interface)

Extend your emulator with a GUI, which allows the user to select a binary to be executed, reset the state of the IJVM instance and enter input and read output (see WebIJVM or IJVMore for inspiration). There are many libraries that allow you to implement a GUI in C, such as GTK, Nuklear and raygui. Remember to document (in the README) the libraries you used and how to install them.

Put your GUI in `src/gui.c`, you can build this using `make gui`. Make a file called `gui_libs.txt` in the root folder of your project (you are not allowed to change the Makefile, as codegrade replaces it with the original one). In this you can put addditional compiler command line arguments (for example `-lgtk-4` for linking with gtk-4), these will be included when running `make gui`. Make sure your file does not end in an enter.

Possible features (pick at least 3):

- Allow the user to select a binary to be executed
- Allow the user to reset the state of the IJVM instance and restart the execution
- Allow the user to enter input in a text field and display the output
- Allow the user to select a file to be as input instead of waiting for keystrokes
- Allow the user to save the IJVM output to a file of choice
- Display information, such as the program counter and stack contents
- Make your GUI portable, so that it works on other operating systems

Debugger

Extend your emulator with an interactive memory debugger with roughly the same functionality as GDB. The program should start on the command line and give the user a prompt. Your debugger should have the following commands:

- `help` : print help on how to use the debugger
- `file <binary>` : load the specified binary
- `run` : run until the next breakpoint is encountered. If the program has already started, stop and start again.
- `input <file>` : set the IJVM input to contain contents of specified file
- `break <addr>` : set a breakpoint for instruction at address addr
- `step` : perform one instruction
- `continue` : continue executing until the next breakpoint
- `info` : show the local stack (in hexadecimal) and variables of the current frame
- `backtrace` : show a call stack of all frames (i.e., in which order methods have called each other and with what arguments)

Create a new program, **debugger**, that reads commands from the command-line, and starts up a new IJVM instance that can be debugged when the run command is executed. You are allowed to use the **readline** library for command line parsing.

Put your debugger in `src/debugger.c`, you can build this using `make debugger`. Make a file called `debugger_libs.txt` in the root folder of your project (you are not allowed to change the Makefile, as codegrade replaces it with the original one). In this you can put addditional compiler command line arguments (for example `-lgtk-4` for linking with gtk-4), these will be included when running `make debugger`. Make sure your file does not end in an enter.

Look at how the available commands map to the interface specified in **ijvm.h** for an idea on how to start implementing a debugger.

Debug symbols

Extend your debugger with debug symbol handling. The goJASM assembler can add debug symbols to a binary. Debug symbols allow you to break the execution at a certain method or at a certain label. Extend the `break` command such that besides an address, you could also provide a label or a method to break at (e.g., `break myfunc`, should break the execution when the method `myfunc` starts executing).

The debug symbols for methods and labels can be found in the third and fourth block of the binary respectively. The format of these sections is as follows:

```
debugblock = [entry]*
entry = <32-bit instruction address> <symbol name> <null terminator>
symbol name = [char]+char = <any ASCII letter>
null terminator = '\0'
```

For example, suppose you have a debug section with an entry with address `0x1337` and symbol name `l33tfunction`. If you execute `break l33tfunction`, you should break at address `0x1337`. The debug symbols for labels should be handled in the same manner. Since different methods can have labels with the same identifier, the assembler automatically prepends the method name to the identifier of labels (e.g., label `L1` in `myfunc` becomes `myfunc#L1`).

Glossary

- **ASCII** a standard for encoding characters in memory. ASCII characters have values in the range 0 to 255.
- **assembly** the act of translating human-readable assembly into machine code to be executed by the computer or virtual machine.
- **binary** a (possibly) non-human readable file format consisting of bytes. Usually the individual bytes have to be combined into integers to make sense of the contents.
- **branch** a branch is an instruction that can divert the control flow of the program, deviating from executing the program instructions in order.
- **byte** a sequence of bits, signifying the smallest unit of addressable memory. On modern computer architectures a byte consists of 8 bits.
- **endianness** the order in which the bytes are arranged in a numerical datatype. big-endian has the most significant bit first, while little-endian has the least-significant bit first.
- **hexadecimal** a way of notating a number. Rather than using base 10 (as in decimal), hexadecimal uses base 16. Usually used for byte notation, as two hex digits can represent one byte.
- **local frame** a piece of memory allocated for a certain subroutine (function). The local frame contains information about local variables as well as the stack used in the subroutine.
- **pointer** a memory object (e.g., 64 bit number) which refers to a certain memory location. Accessing the pointed-to object using the pointer is called dereferencing a pointer.
- **stack** a linear last-in-first-out (LIFO) datatype with the following operations: `top`, `pop`, `push`, `size`.
- **stack machine** a machine architecture that operates on a stack, rather than on registers.
- **word** a data type represented as a four-byte big-endian integer.